Springleik: A Case Study in Small Languages

Mark S. Williamsen

August 30, 2026

Abstract

Small languages arise in computing for a variety of reasons. A simple command prompt can be useful in bringing up new hardware, particularly on small targets running embedded firmware. This leads naturally to a scriptable interface. Which can then evolve further into a primitive domain-specific language (DSL) that supports use cases in development, test, and even production. This work describes one path through the steps that lead from command prompt to small language. Such languages can be very efficient with the resources they use including memory, processor bandwidth, and system services. Memory allocations can be static or dynamic, according to the details of the application. Additional features, including self-documenting code and generation of baseline test outputs, are easily implemented with minimal effort.

1 Introduction

Software developers often assume that the first implementation milestone for an interpreted language is a parser capable of translating human-readable source code into an internal representation. This assumption is reasonable for general-purpose programming languages, where source code is the primary means by which programs are created and maintained. However, it is not necessarily true for small domain-specific languages developed for a single application or product.

This paper explores an alternative development strategy. Rather than beginning with a parser, the executable representation is constructed directly in memory using ordinary programming language facilities. The resulting structure is then executed, verified, serialized, analyzed, and ultimately deployed without ever interpreting a source file. Human-readable representations are treated as derived views of the underlying execution model rather than as the primary artifact.

This approach offers several practical advantages. Interpreter development, command semantics, verification, testing, serialization, visualization, and documentation can proceed independently of the considerably different task of deserializing structured text. In our experience, this separation shortens the path to a deployable system while encouraging a clear distinction between execution and analysis. A parser remains valuable, but its implementation can be deferred until there is a genuine operational need for humans or external tools to author executable descriptions directly.

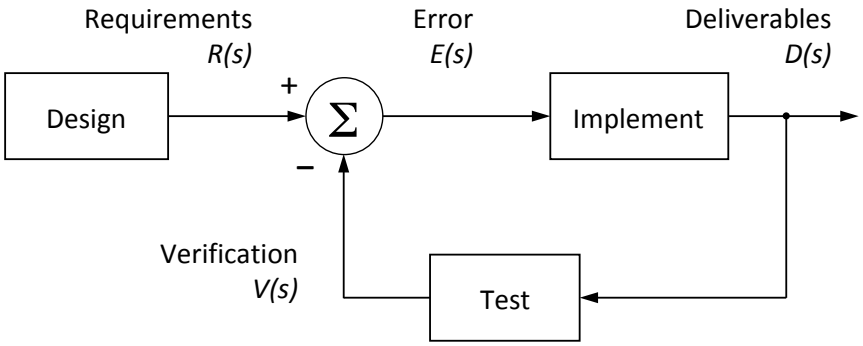

Figure 1: Software development process rendered as a closed-loop simulation diagram. Negative feedback from verification $V(s)$ causes deliverables $D(s)$ to evolve, tracking changes in requirements $R(s)$. When all requirements have been met, the error term $E(s)$ goes to zero and the control loop is in steady state.

The central claim of this paper is therefore not that parsing is unnecessary, but that it need not be the first step. For many bespoke interpreted languages, a complete and useful product can be designed, implemented, verified, and shipped before a source-language parser is ever written.

2 Model of software development

Many models are available for software development for this discussion. Here we present a simple model based on closed loop control theory, which includes concepts of time dependence and iteration. Figure 1 is a simulation diagram representing a software development process. The input on the left is requirements $R(s)$, and the output on the right is executable code and other deliverable assets $D(s)$. In between we have a means for creating code *Implement* positioned as the plant in control theory, and a means for verifying code *Test* positioned as the sensor. The summing junction on the left compares verification results $V(s)$ with requirements $R(s)$ to produce an error term $E(s)$, which then drives ongoing implementation. The directed edges connecting these blocks represent N-dimensional vectors where N is the number of requirements. The value of each dimension is the extent to which a given requirement has been met on some arbitrary scale.

A typical software project will begin with all vectors pointing to the origin: no requirements, no implementation, and no tests. As development proceeds we add requirements which drive the implementation, which is then tested and compared with requirements. Each directed edge conveys important information, but the most interesting vector is the error term $E(s)$. This will wander in an N-dimensional problem space, with N steadily increasing as development proceeds. Ideally this vector should always trend towards zero, with distance from the origin giving us an indication of how far we are from done. Time dependence is not shown explicitly but one can imagine one or more integrators in the loop, with time constants we may or may not control, modifying the time

domain response of the development process.

Treating the development process as continuous we can write an expression for deliverables $D(s)$ in terms of implementation $I(s)$, test $T(s)$, and requirements $R(s)$, where s is the Laplace variable.

$$D(s) = R(s)\frac{I(s)}{1 + I(s)T(s)} \tag{1}$$

In this model we see that there are three things which matter: requirements, implementation, and tests. These pillars of software development should receive equal priority during development, so that all three can be considered as correct and complete when development ends and maintenance begins. As a practical matter however many projects end with one or more of the pillars incomplete. We claim without proof that given any one of these, the other two can be derived through various means. For the purposes of this paper we use the simulation diagram in figure 1 to define deriving implementation and tests from requirements as "forward engineering." Starting with an implementation or a test suite is then defined as "reverse engineering."

3 Object model for small targets

Developers of embedded software for small targets may be tempted to organize their object model around things the system has, such as inputs, outputs, sensors, and actuators. We submit however that a coherent design is more likely to result from an object model organized around things the embedded system does, such as "home an axis", "measure a sample", or "close a relay." The reason we think in terms of objects is so that we can throw them into a container and iterate over them. While there may be a use case for iterating over peripherals, the important use case here is iterating over a collection of nodes which represent actions a user might want to invoke. These are packaged as command nodes along with the arguments needed to carry out each action. Thus we replace a noun-oriented design with a verb-oriented design so that command nodes can be collected into sequences which can then be interpreted on live hardware, simulated hardware, or a hybrid of both. Let's look at an example of how such a design might arise.

Consider a small target device with embedded processor and several axes of motion control. On first bring-up we want to start characterizing the motion controller. We write a small monitor program to run on the target with a command prompt exposed on a serial port, or a socket port if so equipped. Then we write commands to initialize the motion controller, home an axis, move axis to a new position, and wait for position reached. Each command can be followed with one or more arguments giving details such as velocity, acceleration, current, position, etc. Running a terminal program on a host computer allows us to manually exercise various behaviors of the motion controller. We can stream a text file containing motion commands from the host to the target's monitor program to execute a motion sequence of arbitrary complexity. Move commands however take finite time, and we wind up overrunning the monitor program with new commands before old ones are complete. So we write a program to run on the host and carry out a series of motions on the target using monitor program

commands, but now waiting for a prompt from the target before sending a new command from the host.

4 Minimum footprint

Many readers will have gone through these steps in bringing up a new target. We don't yet have a small language, but many of the pieces needed are already in place. Now we implement our object model by declaring a class object for each command and an instance attribute for each argument the command needs. One can also add an instance attribute for the command's return value, if that value is meaningful. Now we have a small language we can use to carry out the following steps:

- Instantiate one or more class objects representing command nodes to be executed.
- Populate the nodes with arguments appropriate for the desired commands.
- Compose the nodes into a sequence by appending them to an ordered collection.
- Iterate over the collection, interpreting each node as a command to be sent to the target.
- Capture results returned from each command, either as attributes of the command nodes or in a separate trace or listing.

Let's look at what we have. The class objects representing command nodes can live almost anywhere: on the host, on the target, in the cloud, or anywhere else that can interpret them into commands with arguments. For now we think of the arguments as being hard-coded, but they could just as well be derived or symbolic. Various types of collections can be used to iterate over the command nodes, as long as the iteration follows a predictable, repeatable path. The commands themselves can be sent to one or more target devices, or to any other device or simulation with an interface to receive them. Command replies such as motion results or error messages in our motion control example can go back to the host, to some other monitor, or even be ignored. This then is the bare minimum of what's needed to claim we have a small domain-specific language. This can be prototyped and running in a matter of a few days, completely avoiding the trap that "nothing works until everything works."

5 Serialization

Now let's look at what's missing. To obtain a text representation of the program we'll need a means to serialize the collection of command nodes. We have many well-established choices such as JSON, XML, YAML, etc. We also have the freedom to design our own representation if warranted, or dispense with source code altogether. In this paper we will assume without loss of generality that JSON has been chosen. Experience shows that it is trivial to give command node classes a serialize method which is specialized for specific commands and

```
[
  {"cmd": "initControl"},
  {"cmd": "setRunCurrent", "axis": 1, "current_mA": 500},
  {"cmd": "setHoldCurrent", "axis": 1, "current_mA": 500},
  {"cmd": "setVelocity", "axis": 1, "steps_sec": 10},
  {"cmd": "setAccel", "axis": 1, "steps_sec2": 10},
  {"cmd": "homeAxis", "axis": 1},
  {"cmd": "waitPosition", "axis": 1},
  {"cmd": "setPosition", "axis": 1, "degrees": 200},
  {"cmd": "waitPosition", "axis": 1},
  {"cmd": "setPosition", "axis": 1, "degrees": 0},
  {"cmd": "waitPosition", "axis": 1},
  {"cmd": "setPosition", "axis": 1}
]
```

Figure 2: JSON representation of a simple command list being sent to an embedded motion controller. Arguments here are named, but could also be positional. JSON text copied from file *two.json* in the Springleik project repository on GitHub.

arguments. To obtain a program listing we iterate over the collection while serializing each node. If we have multiple interpreters (host, target, cloud, etc.) all should produce identical program listings for a given collection of command nodes. Returning to the example of an embedded motion controller we can look at what the serialized JSON representation might look like in figure 2. It should be mentioned here that while off-target interpreters can use the monitor command prompt interface, on-target interpreters don't have to since they can call directly into functions on the target.

6 Control nodes

Real programming however implies branching and looping, which is easily added by defining control nodes for conditional and repeated execution where the arguments represent loop count, limits for comparison, etc. At this point we've only described programs which are a simple list or sequence of nodes. Now we need to expand our definition of command nodes to include a node list attribute, allowing us to represent composite tree structures. But not every node needs to have a list. So we divide node classes into composite or *branch* nodes which have a list, and terminal or *leaf* nodes which don't. Clearly conditional and repeating nodes should have lists, to which we append nodes that should only be executed if the condition evaluates as true, or which will be repeated in iterations controlled by a repeating node. Our class hierarchy can begin with the base class *node* which has two subclasses, *branch* and *leaf*. Command and control nodes can then be derived from *branch* and *leaf*. All node classes should have methods to execute and to serialize them while traversing a command tree by recursive descent. The execute and serialize methods of composite nodes must also iterate over the node list, calling the execute or serialize method of each subordinate node in turn. The execute method of a composite node can include entry and exit code, to run before and after iterating over the node list.

```
{"cmd": "initControl", "list": [
  {"cmd": "setRunCurrent", "axis": 1, "current_mA": 500},
  {"cmd": "homeAxis", "axis": 1},
  {"cmd": "setVelocity", "axis": 1, "steps_sec": 10},
  {"cmd": "setAccel", "axis": 1, "steps_sec2": 10},
  {"cmd": "setRunCurrent", "axis": 2, "current_mA": 500},
  {"cmd": "homeAxis", "axis": 2},
  {"cmd": "setVelocity", "axis": 2, "steps_sec": 10},
  {"cmd": "setAccel", "axis": 2, "steps_sec2": 10},
  {"cmd": "iterateRegister", "reg": 0, "start": 0, "stop": 200,
    "step": 20, "list": [
    {"cmd": "setPosition", "axis": 1, "degrees": "reg"},
    {"cmd": "waitPosition", "axis": 1},
    {"cmd": "iterateRegister", "reg": 0, "start": 0, "stop":
      100, "step": 10, "list": [
      {"cmd": "setPosition", "axis": 2, "degrees": "reg"},
      {"cmd": "waitPosition", "axis": 2}]}]},
  {"cmd": "testFlag", "flag": 1, "trueIf": 1, "list": [
    {"cmd": "setHoldCurrent", "axis": 1, "current_mA": 50},
    {"cmd": "setHoldCurrent", "axis": 2, "current_mA": 25}]}]}
```

Figure 3: JSON representation of a command tree with conditional and repeating nodes. The root node is both a dictionary and a command node. But we could just as well have subclassed *branch* to have a *root* node class. This is a design decision, along with many other details. JSON text copied from file *three.json* in the Springleik project repository on GitHub.

In figure 3 we look at the JSON representation of a command tree which includes conditional and repeating nodes. Iteration is done by the new "iterateRegister" control node that changes the value of a register attribute which is visible to nodes beneath the control node. Nested loops are supported here by placing the iteration of axis 2 inside the iteration of axis 1. At the end of the sequence a "testFlag" control node tests a flag to check whether to return to holding current.

The ability to implement tree structures composed of command and control nodes, using all of the infrastructure we've developed so far, brings into focus a scalable, portable, and verifiable framework for representing interpreted programs optimized for the application at hand. The composite tree structure maps directly to structured text representations such as JSON, XML, or YAML. Human-readable program listings are easily obtained by serializing nodes while traversing the tree by recursive descent. In this picture we can now say clearly that command nodes are dictionaries composed of key-value pairs representing the command name and all of its arguments. Program nodes can also contain an ordered list identifying subordinate nodes.

7 Memory allocation

Command tree programs which rarely change can be composed of statically allocated nodes. Programs that could change at run-time however will be com-

```
    # composite command tree from figure 3
    elif index == 3:
        node.tree = initControl()
        node.tree.append(setRunCurrent(1, 500))
        node.tree.append(homeAxis(1))
        node.tree.append(setVelocity(1, 10))
        node.tree.append(setAccel(1, 10))
        node.tree.append(setRunCurrent(2, 500))
        node.tree.append(homeAxis(2))
        node.tree.append(setVelocity(2, 10))
        node.tree.append(setAccel(2, 10))
        outerLoop = iterateRegister(0, 200, 20)
        node.tree.append(outerLoop)
        outerLoop.append(setPosition(1, 'reg'))
        outerLoop.append(waitPosition(1))
        innerLoop = iterateRegister(0, 100, 10)
        outerLoop.append(innerLoop)
        innerLoop.append(setPosition(2, 'reg'))
        innerLoop.append(waitPosition(2))
        testBranch = testFlag(1, 1)
        node.tree.append(testBranch)
        testBranch.append(setHoldCurrent(1, 50))
        testBranch.append(setHoldCurrent(2,25))
```

Figure 4: Python code snippet from function *figureFactory* in file *host.py* in the Springleik project repository on GitHub. Shown are the steps to instantiate and compose the command tree serialized in figure 3.

posed of nodes taken from a node pool, or using dynamically allocated memory. In garbage-collected environments dynamic nodes will evaporate after they are no longer referenced. Releasing tree nodes without garbage collection is easy since we can ask the root node to delete itself and all subordinates recursively. For this to work all nodes in the tree should be dynamically allocated and each node should appear in the tree exactly once. It's easy to maintain a global node counter to check for leakage (block not freed) and double frees (block freed twice) at run-time. An example of this appears in the variable *nodeCount* in file *asynTree.c* in the Springleik project repository on GitHub.

In figure 4 we show an example of instantiating and composing a command tree in a Python function which implements a tree factory. That combined with a serialization function gives us a human-readable text representation of the command tree program as structured text. In other words, the interpreter source code is written for us, and we can put off writing the source code parser as long as we like.

One can instrument the command tree by adding node attributes like time stamps, tally counters, iterator values, and reply strings. When added to serialization functions, these attributes then appear in the structured text source code with each attribute conveniently attached to the node where it originated. We describe this as "decorating the tree." Serializing the tree before and after execution shows clearly where the active paths lie. If memory permits, one can consider keeping time or position series data in the same way, appending new

data points to a list maintained by the node where the data points originated. In motion control applications, for instance, one can record axis position while polling for motion done. Such data nodes can be given a serialization function, while their execution function should be a simple pass-through.

8 Analysis

In this picture capturing data points like shaft position will be fairly light-weight, perhaps just reading a shaft encoder. Ideally this can be done with no impact on system performance. However in some cases we will want to perform further analysis, for instance extracting fitted parameters like velocity and acceleration from time series data. Since the raw data exists in any serialization done after execution, we can perform analysis wherever the serialized command tree exists. An obvious choice would be to run analysis in the same process that runs the interpreter. In other words we take a tree which already supports execution and serialization, and then add an analysis method, all traversing the tree by recursive descent. Alternately one can insert specialized analysis nodes into the tree, for the purpose of analyzing data nodes beneath them. Here we encounter the principle of *Separation of Execution and Analysis*. This will be discussed further in section 9.

One might wonder were to put the results of analysis, and we now propose that the best place is to append analysis nodes to the command tree, just as we previously appended data nodes. So any serialization of the tree done after analysis will contain the analyzed results attached to the nodes where they originated. Some applications will require multiple passes of analysis, which is easily accomodated in this technique by writing several analysis functions to be called in sequence. This is a design decision which needn't be taken until analysis is needed.

Reporting in any desired format can also be carried out by recursive descent, facilitated by inserting report nodes or adding a report method to existing node classes. Reporting can take many forms, such as tables, charts, and graphs. Now we see the pattern, that every step in the workflow can be thought of as traversing a command tree by recursive descent in a series of passes. With each pass performing the desired operation and appending the results, in a process which is scalable and verifiable across multiple levels.

It is especially convenient to carry out syntax and integrity checks when required in a given application. One can establish rules as to which node classes can be subordinate to which other node classes, set limits on attribute values, check that every node is appended exactly once (no cycles), and so forth. Integrity can be verified by computing a hash over the tree. If data nodes are included in the hash, it can be used to detect new data to be uploaded. To summarize, we list here some possible workflow steps that will be both possible and practical in this technique. In general these can be invoked in any order against the whole tree or any subtree within it, thus establishing a command tree lifecycle:

- Instatiate – construct or allocate nodes needed for the initial tree,
- Compose – establish parent/child relationships between all nodes, extending in a branching structure from a single root node,

- Serialize – render command tree as structured text,
- Verify – check syntax and verify integrity according to the needs of the application,
- Execute – carry out a program of interactions with hardware and communications,
- Analyze – gather and summarize data found in subordinate data nodes,
- Report – render formatted reports of gathered data and analysis nodes,
- Release – deallocate or free memory used for the command tree and all subordinate nodes.

9 Deserialization

There are several features we take for granted in established programming languages which might not be needed in a small bespoke language. One is deserialization, used to parse a structured text representation into nodes or tokens in memory. Another is the ability to parse and execute general math and logic expressions. Experience shows that these features impose a burden of time and expense to implement, document, and verify. The techniques described here give one the choice of when and how to implement additional features, according to the needs of the project.

There are several use cases which justify the need for deserialization. In our view the most important of these is testability, to allow for round-trip testing. We propose here that since serialization is so much easier than deserialization, serialization should be implemented and verified first to use as a trusted reference. Which in turn makes the deserializer, that is the source code parser, easier to implement and verify. Archived output from the serializer can be deserialized and then serialized again. With the new code being compared to the archived code for equality. This will be discussed more in section 11.

Because the deserializer isn't needed to create and execute AST command trees, it can be implemented class by class, as and when needed. The way this works is that we begin with all source code nodes deserializing to the default base classes for leaf and branch. Then we add specialized implementation for each node class in turn. The deserializer is typically a factory method which instantiates and composes the nodes encountered while parsing source text. As the deserializer starts working, we can begin to add test cases which rely on it. Specifically, we can have tests where instead of executing the tree on live hardware to populate it with data nodes, we can deserialize an archived command tree which already has data nodes attached. Thus removing the dependence on live hardware for all steps in the workflow which come after execution. This principle of *Separation of Execution and Analysis* opens up many possibilities for testing on-target and off-target, and for verifying with test cases obtained directly from archived live data, live data modified to stress limits and edge cases, synthetic data meant to prove mathematical robustness, or hybrid combinations of these sources.

A secondary use of deserialization is to move data analysis and reporting off-target to host software running on a workstation or in the cloud. Here

too one only needs partial deserialization to support analysis and reporting, leaving hardware-dependent nodes as pass-through base class nodes. This is especially helpful in applications with multiple stages of analysis, or multiple report formats. All can be tested with the same source data.

As a practical matter, experience shows that implementation of deserialization is best accomplished in two steps. First create a means for deserializing structured text such as a JSON file into a composite tree structure in memory, and providing a serialization method to allow round-trip verification of the structured text parser. Then create a separate means for rendering the composite tree structure in memory into an interpreter command tree. The benefit in this approach is that the structured text parser can enforce all of the rules of the text representation, while not needing any domain-specific knowledge defined by the application. The rendering component can then be assured of receiving valid inputs, while enforcing domain-specific rules and requirements. Now we can use the previously verified command tree serialization method to obtain a structured text representation to compare with the deserializer input.

10 Math and logic expressions

The work described here has its origins in the logic scan of industrial programmable logic controllers. Having said that, we will only discuss math and logic expressions in an overview. Evaluation of such expressions by recursive descent implies subclasses of nodes which return math and logic values when executed. Leaf (terminal) nodes would return a value obtained from hardware such as analog and digital inputs. Leaf nodes could also return values from internal registers as might be used in communicating with other subsystems. Branch (composite) node classes would be provided for the operations of boolean logic, and for those math operations needed for the application.

The ability to parse general math and logic expressions is probably not a requirement for most embedded interpreters, especially in domain-specific applications where needed computations can be coded into pre-defined node classes. If however the need for mutable math and logic expressions can't be avoided, we would advocate for adding new execute methods to math and logic leaf nodes which return the needed values. Similarly branch nodes which need those return values can invoke the new execute methods of their subordinates to obtain them. This is clearly a step change in complexity for implementation, verification, and documentation.

11 Asymmetric tree comparison

Performing a directional comparison is important when comparing test outputs with reference files, so that the test files can include additional comments, annotation, and instrumentation. Additionally, items you don't want to compare (timestamps, sequence numbers that change often, analog values such as voltage or temperature, etc.) can be ignored in the comparison by removing them from the reference file. The test file can then have varying values for these items and still be considered as equal, in order to pass a regression test for instance. An interesting question is whether to include version info for the software under

test. If you leave version info in, then you'll have to touch the reference file each time you test a new version. If you actually need an exact match in both directions, simply test the files twice, reversing their order so both are evaluated as test files. If both comparisons are True, then both files have exactly the same dictionary keys and array elements. Following is a list of rules used in this work to compare test outputs with reference files:

- Any dictionary in the test file must have all of the keys in the corresponding dictionary in the reference file. Order is ignored. Dictionaries in the files being compared can have a completely different ordering, and still test as equal. Additional keys in the test file are ignored.
- Any array in the test file must have at least as many elements as the corresponding array in the reference file. Order is maintained and must match, up to the last element in the reference file. Additional array elements in the test file are ignored.
- Each value in a dictionary in the reference file must be equal to the corresponding value in the test file, according to these rules.
- Each element in an array in the reference file must be equal to the corresponding element in the test file, according to these rules.
- Integers must have the same value and sign.
- Floats must have the same value, sign, and exponent. Floats with the value Infinity in the reference file must have a corresponding Infinity in the test file. Sign is not considered in this comparison. Floats with the value NaN in the reference file must have a corresponding NaN in the test file.
- An optional argument can be used to set an error delta. If used, float values will be considered as equal if the absolute value of their difference is less than the error delta value.
- Booleans must either be both True, or both False.
- Text strings must be an exact case-sensitive match, including white space and escape characters inside the string. White space outside of text strings is ignored. The two files being compared can have a completely different formatting in terms of white space, and still test as equal.
- Null values in the reference file must have a corresponding Null value in the test file.

The content of this section has been published previously in reference [10].

12 Execution model

Much of the work described here has been done on targets which support a real-time operating system (RTOS). Since the Springleik command tree carries along with it all of the arguments and parameters needed for execution, all that's needed to initiate a task thread to execute the tree is a pointer or reference to

the root node. On exit the pointer or reference will identify a tree now having updated state information in its attributes, along with any instrumentation and data nodes appended during execution. Having a unique pointer or reference allows the developer to think clearly about the life cycle of each command tree. By limiting access to hardware and global data to specific commands or node classes, it becomes straightforward to implement synchronization primitives to maintain thread safety between command trees.

Springleik command trees easily support cooperative multitasking by inserting appropriate thread waits. While one could introduce a node class to do this it's easier to put waits into control node execute methods, to relinquish control between calls to subordinate nodes.

13 Reuse and granularity

One may well wonder how much of this can be reused across multiple projects. Our experience has shown that command nodes are clearly domain specific and can only be reused for very similar applications, while control nodes are not domain specific and can be reused freely. The resulting plethora of small languages can be justified by our need to maintain a small footprint by optimizing the command tree language for each application.

A related question is how to set the granularity of command and control nodes. We propose starting with one command node class for each monitor command on the target, and one control node class for each branching or gating operation. If math and logic expressions are to be supported, start with one math or logic operation per control node. As development progresses common patterns will soon become apparent, which then become candidates for integration into more functional node classes. These are design decisions which don't have to be decided up front.

14 Toy model

The Springleik repository on GitHub includes a toy simulation of a three axis motion controller which implements a target process and a host process which communicate through a TCP socket port. As described in the *ReadMe* file both processes expose a console prompt command line. The target process command line accepts commands related to the simulated hardware. The host process command line accepts commands related to the abstract syntax tree interpreter. Any commands the host doesn't recognize are passed along to the connected target process. The target process uses TkInter to render the shaft position indicators shown in figure 5. The host process includes a command tree factory method which can instantiate and compose the structures given here in figures 2 and 3.

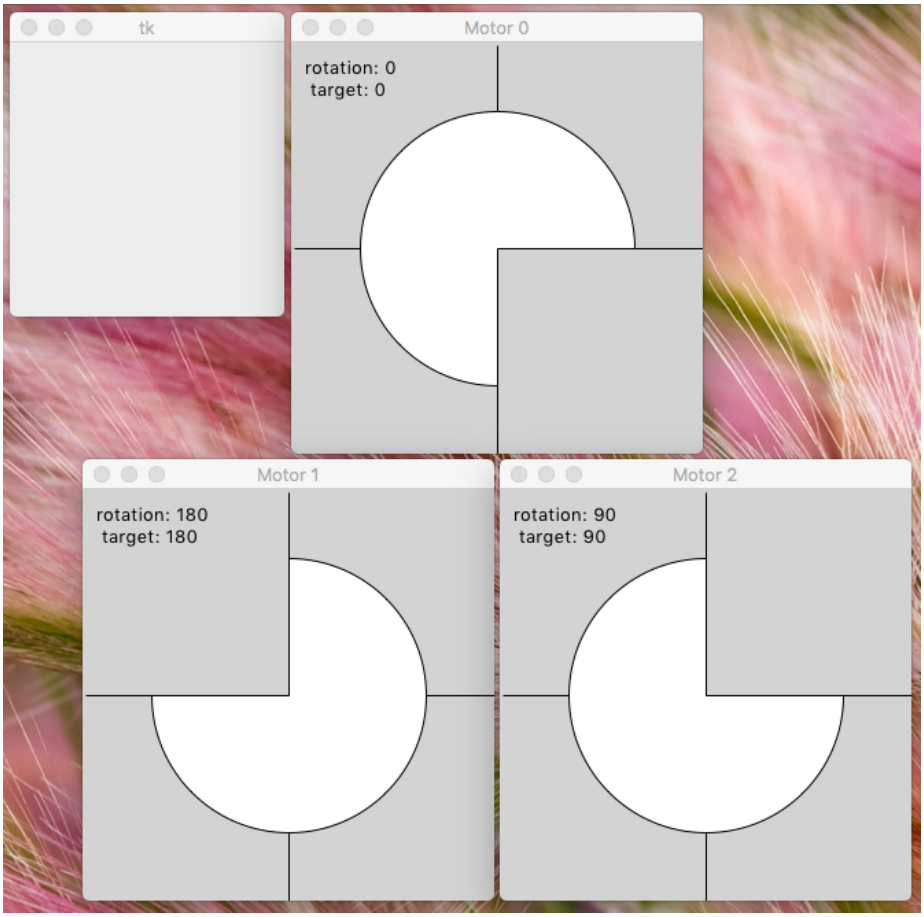

Figure 5: Three axis motion control simulation implemented in a target process. A separate host process implements an abstract syntax tree (AST) interpreter using Springleik techniques. Source code is posted in the Springleik project repository on GitHub.

References

[1] A Pattern Language (Christopher Alexander 1977)

[2] Notes on the Synthesis of Form (Christopher Alexander)

[3] Design Patterns (Gang of Four 1994)

[4] MSOE textbook on Control Systems Theory

[5] MSOE textbook on Operating Systems

[6] Books or papers by Stroustrup, for instance my favorite book on C++

[7] Anything useful by Donald Knuth, Niklaus Wirth, or Edsger Dijkstra

[8] Springleik project repository on GitHub

[9] Find a review paper about Merkle trees

[10] PompTree repo with ReadMe giving rules of asymmetric tree comparison